

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

Álgebra lineal aplicada a IA

Módulo 2 · Matemáticas y Fundamentos Técnicos de IA

Vectores, matrices, similitud coseno, PCA y SVD: la maquinaria matemática detrás de embeddings, RAG y LoRA.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Álgebra lineal aplicada a IA

Cuando una API de OpenAI devuelve un embedding de 1536 dimensiones para una frase, cuando un sistema RAG calcula qué documento es más relevante para una consulta, o cuando un transformer procesa una secuencia de tokens mediante el mecanismo de atención, todo eso es álgebra lineal en acción. No es matemática abstracta: es la maquinaria que hace funcionar cada sistema de IA que usas a diario.

La buena noticia es que no necesitas ser un matemático experto para entender el álgebra lineal que importa en IA. Necesitas dominar cuatro conceptos: vectores y operaciones entre ellos, matrices y multiplicación matricial, espacios vectoriales y proyecciones, y descomposiciones como PCA y SVD. Con esos cuatro pilares puedes entender embeddings, atención, fine-tuning eficiente (LoRA) y búsqueda semántica.

Los cuatro pilares del álgebra lineal para IA

1. Vectores: representar el significado como geometría

Un vector es una lista ordenada de números que representa un punto en un espacio multidimensional. En IA, los vectores son la forma fundamental de representar cualquier entidad: una palabra, una frase, un documento, una imagen, un usuario, un producto. La clave es que la geometría del espacio vectorial captura relaciones semánticas: cosas con significados similares están cerca en el espacio, cosas con significados opuestos están lejos o en direcciones opuestas.

La similitud entre dos vectores se mide con la similitud coseno: $\cos(\theta) = (a \cdot b) / (||a|| \times ||b||)$, donde $a \cdot b$ es el producto punto (suma de productos elemento a elemento) y $||a||$, $||b||$ son las normas (longitudes) de los vectores. Esta fórmula produce un valor entre -1 (opuestos) y 1 (idénticos). Es exactamente la operación que ejecutan las bases de datos vectoriales como Pinecone, Milvus y FAISS cuando buscas los K documentos más similares a una consulta.

2. Matrices: transformar representaciones

Una matriz es un arreglo rectangular de números que representa una transformación lineal: toma un vector y lo mapea a otro vector, posiblemente en un espacio de diferente dimensión. En una red neuronal, cada capa aplica una transformación matricial: $\text{output} = \text{activation}(W \cdot \text{input} + b)$, donde W es la matriz de pesos, b es el vector de sesgos, y activation es una función no lineal. El entrenamiento es el proceso de encontrar los valores de W y b que minimizan la pérdida.

El mecanismo de atención QKV es tres multiplicaciones matriciales en secuencia: $Q = X \cdot W_Q$, $K = X \cdot W_K$, $V = X \cdot W_V$. Los pesos W_Q , W_K , W_V se aprenden durante el entrenamiento. LoRA (Low-Rank Adaptation), la técnica de fine-tuning más usada en empresa, funciona aproximando las actualizaciones de los pesos como el producto de dos matrices de bajo rango: $\Delta W \approx A \cdot B$, donde A y B son matrices pequeñas. Esto reduce los parámetros entrenables de

cientos de millones a decenas de miles.

3. Espacios vectoriales y proyecciones

Un espacio vectorial de d dimensiones es el conjunto de todos los vectores de d componentes. Los embeddings de OpenAI text-embedding-3-small tienen 1536 dimensiones; los de text-embedding-3-large tienen 3072. Estas dimensiones no tienen interpretación semántica individual (no es que la dimensión 42 mida "importancia" o la dimensión 100 mida "positividad"): las relaciones semánticas emergen de la estructura geométrica del espacio completo.

La proyección es la operación de encontrar el punto más cercano de un vector sobre un subespacio. Se usa en PCA para reducir dimensionalidad proyectando los datos sobre las direcciones de máxima varianza. También aparece en búsqueda aproximada de vecinos: en lugar de calcular la distancia exacta a todos los vectores del índice (costoso en millones de documentos), se proyectan los vectores sobre subconjuntos y se aplica búsqueda exacta solo en los candidatos más prometedores.

4. PCA y SVD: compresión con preservación de estructura

La Descomposición en Valores Singulares (SVD) factoriza cualquier matriz M como $M = U \cdot \Sigma \cdot V^T$, donde U y V son matrices ortogonales y Σ es una matriz diagonal con los valores singulares. Los valores singulares más grandes capturan la mayor parte de la información; los más pequeños son ruido. Truncar la SVD a los k valores singulares mayores produce la mejor aproximación de rango k de la matriz original. PCA es un caso especial de SVD aplicado a la matriz de covarianza de los datos.

SECCIÓN 3 · CÓMO FUNCIONA

Álgebra lineal en el ciclo completo de un sistema RAG

Paso 1: indexación. Cada documento se convierte en un vector de embedding mediante un modelo como text-embedding-3-small. Internamente, el modelo pasa el texto por un Transformer encoder y extrae el vector de la capa final. Este vector es el resultado de múltiples multiplicaciones matriciales y operaciones de atención. Se almacena en el índice vectorial (FAISS, Pinecone, Weaviate).

Paso 2: búsqueda. La consulta del usuario se convierte también en un vector de embedding. El índice vectorial calcula la similitud coseno entre el vector de consulta y todos los vectores indexados. Los k vectores más similares corresponden a los documentos más relevantes. Esta operación es álgebra lineal pura: multiplicaciones vectoriales y cálculos de norma.

Paso 3: generación. Los documentos recuperados y la consulta del usuario se incluyen en el contexto del LLM. El LLM procesa el contexto mediante el Transformer: capas de multiplicaciones matriciales y operaciones de atención, todas álgebra lineal. La respuesta final emerge de una proyección sobre el vocabulario seguida de softmax.

Cada uno de estos pasos es, en el fondo, operaciones sobre vectores y matrices. La diferencia entre un sistema RAG eficiente y uno lento o costoso está en la dimensionalidad de los embeddings (más dimensiones = más precisión pero más coste de almacenamiento y

búsqueda), el método de indexación (exacto vs aproximado), y la estrategia de proyección para la búsqueda aproximada.

SECCIÓN 4 · EJEMPLOS REALES

- OpenAI text-embedding-3-small: 1536 dimensiones, coste de \$0.02/M tokens, usado en millones de sistemas RAG. El aumento de dimensionalidad de 1536 a 3072 (text-embedding-3-large) mejora la recuperación en ~5-10% en benchmarks MTEB a cambio de doblar el coste de almacenamiento y cómputo.
- FAISS (Facebook AI Similarity Search): librería open source que implementa búsqueda de vecinos aproximados con índices IVF (Inverted File Index) y PQ (Product Quantization), reduciendo el espacio de búsqueda de $O(n)$ a $O(n/k)$ con pérdida de recall controlada. Usada en sistemas de búsqueda semántica de escala masiva.
- LoRA aplicado a Llama 3 70B: fine-tuning de un modelo de 70.000 millones de parámetros con solo ~5-50 millones de parámetros entrenables (rango $r=16$ en las matrices de atención). Posible gracias a la aproximación de bajo rango que captura las actualizaciones relevantes en un subespacio pequeño.
- PCA en clustering de tickets: reducir embeddings de 1536 dimensiones a 50 dimensiones con PCA antes de clustering con k-means. Reduce coste computacional y elimina ruido de dimensiones de baja varianza, produciendo clusters más coherentes en muchos casos prácticos.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: "Más dimensiones de embedding siempre es mejor." No. A partir de cierto punto, añadir dimensiones añade ruido y coste sin mejorar la recuperación. La elección del modelo de embedding debe basarse en benchmarks específicos para el dominio y el idioma, no en el número de dimensiones.

Error 2: "La similitud coseno mide lo mismo que la distancia euclídea." No exactamente. La similitud coseno mide la orientación (ángulo) entre vectores, ignorando la magnitud. La distancia euclídea mide la distancia absoluta. Para embeddings normalizados (norma = 1) ambas son equivalentes, pero para embeddings no normalizados producen rankings diferentes.

Error 3: "La búsqueda vectorial exacta es siempre mejor que la aproximada." Depende del caso de uso. La búsqueda exacta es $O(n)$ en el número de documentos; a millones de documentos con latencias de milisegundos, es impracticable. Los índices aproximados (HNSW, IVF) producen el 95-99% del recall de la búsqueda exacta con latencias 10-100x menores.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

La aplicación práctica más directa es implementar un sistema de búsqueda semántica desde cero con NumPy para entender la mecánica antes de usar librerías de alto nivel. Los pasos: generar embeddings para un conjunto de documentos (con la API de OpenAI o un modelo local de Hugging Face), almacenarlos en una matriz NumPy, calcular la similitud coseno entre un vector de consulta y todos los vectores del índice con una multiplicación matricial, y ordenar por

similitud. Todo esto es menos de 20 líneas de NumPy y construye una comprensión sólida de lo que hacen FAISS o Pinecone internamente.

Para sistemas de producción, la elección del índice vectorial depende de: número de documentos (hasta ~100K: FAISS en memoria; hasta ~10M: FAISS con IVF; más de ~10M: soluciones gestionadas como Pinecone o Weaviate), frecuencia de actualizaciones (índices exactos son más fáciles de actualizar; índices aproximados requieren reindexación parcial), y requisitos de latencia vs recall.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M1-T8/T9 (Transformer y atención): el mecanismo QKV y la self-attention son multiplicaciones matriciales. Sin álgebra lineal, estos mecanismos son cajas negras.
- M7 (RAG): la búsqueda semántica es álgebra lineal sobre vectores de embedding. La elección del modelo de embedding y del índice son decisiones de álgebra lineal aplicada.
- M8 (LLMOps / Fine-tuning): LoRA y QLoRA son técnicas de álgebra lineal de bajo rango aplicadas al fine-tuning eficiente de LLMs.

SECCIÓN 8 · RESUMEN EJECUTIVO

El álgebra lineal es la maquinaria matemática de la IA: vectores (representación), matrices (transformación), similitud coseno (búsqueda semántica), PCA/SVD (compresión y reducción de dimensionalidad). Los embeddings son vectores en espacios de alta dimensión donde la proximidad geométrica captura similitud semántica. La búsqueda semántica es una operación de similitud coseno entre vectores. LoRA es SVD truncada aplicada a actualizaciones de pesos. Dominar estos conceptos permite diseñar sistemas RAG eficientes, evaluar modelos de embedding, optimizar costes de almacenamiento y entender el fine-tuning eficiente.